

From Complete Beginner to Junior ML Engineer: 7-Month Bioinformatics Runway

July 31, 2026 → February 2027 (BSc Hons Conversion + MSc, South Africa)

How this roadmap is built

You're starting from **shaky Python, near-zero SQL, and no formal ML/math background** — but you're a science graduate, which means you already know how to learn hard material rigorously. That's the asset this plan is built around.

Two hard constraints shape everything below: 1. **NYSC is active until early October 2026.** Realistic study capacity during this window is **2.5–3 hrs/day, 5–6 days/week** (~65–80 hrs/month). 2. **From October onward you're free.** Capacity jumps to **5–6 hrs/day, 5–6 days/week** (~130–160 hrs/month), until you leave for South Africa in February 2027.

That gives you roughly **~850–1000 total hours**. That is enough to go from zero to a genuinely hireable junior ML engineer / bioinformatics-lab-ready profile — *if* the difficulty ramps correctly. This plan is phased so you are always working at the edge of your ability, never dropped into the deep end.

Non-negotiables: - 1 full rest day per week, always. - Every phase ends with a **gate checkpoint** — a short list of things you must be able to do before moving on. If you can't, spend 3–5 extra days closing the gap before advancing. This is what prevents the “feeling like an idiot” failure mode: you never move to material that assumes something you don't have yet. - Keep a **weekly log** (even 5 bullet points: what you learned, what you built, what confused you). This becomes both a diagnostic tool and raw material for your CV/personal statement later. - All three portfolio projects are specified in enough detail that you (or another AI assistant) can pick up the spec cold and execute or extend it without re-deriving the plan.

On the hour budgets below: they're written as daily averages *within* a given week's focus, not a literal daily checklist of every subject at once — most phases below are explicitly broken into sub-blocks of 2 weeks each so you're never juggling more than 2–3 subjects on a single day. If you still fall behind, the first things to cut are, in order: Project stretch goals → the R primer in Phase 5 → one Kaggle “Getting Started” competition in Phase 3. The core pipeline requirements of Projects 1–3 and the gate checkpoints are the load-bearing parts of this plan and shouldn't be the first casualty of a bad week.

Phase 0 — August 2026: True Foundations (Python + SQL Intro + Math Intuition)

NYSC active · ~2.5–3 hrs/day · Weeks 1–4

Goal: Stop being intimidated by a blank script, a terminal, or a git command. Build raw comfort with Python syntax and thinking in code, get the SQL mental model in place, get the basic developer toolchain installed and habitual, and build visual/intuitive number sense for the math that’s coming — no rigor yet, just intuition.

Fixed gap: the original draft of this phase had no terminal, git, or environment setup at all, despite every later phase assuming a GitHub repo exists. That’s fixed below — it’s front-loaded into weeks 1–2 so it’s second nature by the time Project 0 needs it in Phase 1.

Fixed gap — the math ramp. The original version of this phase went straight into 3Blue1Brown’s Linear Algebra series on day one. That assumes comfortable footing with function notation, graphing, and symbolic manipulation — things a microbiology degree usually doesn’t build (it builds applied stats intuition, not linear algebra, and general algebra fluency is often rusty a few years out even for strong science grads). Trying to absorb “eigenvector” intuition while also silently relearning what $f(x)$ means is how people bounce off self-study and conclude they’re “bad at math,” when really the sequencing was wrong. Fixed below with a short diagnostic-first ramp — if it turns out algebra is fine, this costs one day, not five.

Days 1–5: Math diagnostic & algebra/notation ramp (do this first, before Linear Algebra)

- **Day 1 (45 min):** take Khan Academy’s **Algebra 1 “Course Challenge”** (a built-in diagnostic — it flags exactly which sub-skills are shaky instead of you guessing). Also skim a short primer on the handful of math symbols ML material uses constantly: summation (Σ), function notation $f(x)$, subscripts/superscripts, and set notation. This alone resolves most of the “notation panic” that makes early ML material feel harder than it is.
- **Days 2–5 (45–60 min/day, only for what the diagnostic flagged):** patch gaps using targeted Khan Academy **Algebra 1 / Algebra 2** units — commonly: manipulating and rearranging equations, exponents and radicals, and Khan Academy **Precalculus’s “Functions”** unit (function notation, reading and sketching graphs, composition). If the diagnostic came back mostly clean, compress this to 1–2 days and move on — don’t manufacture busywork.
- This block replaces that many days’ worth of the “Math intuition” time below — it isn’t additional load, it’s a reordering of the same hours so

Linear Algebra actually lands when you get to it.

Weeks 1–2 (~2.75 hrs/day)

- **Python fundamentals (1.5 hrs/day):** CS50P — Harvard’s “Introduction to Programming with Python” (free, edX/YouTube), problem-set by problem-set. Don’t skip the autograded exercises. Cover: variables, functions, conditionals, loops.
- **Tooling & environment setup (30 min/day):** this is new and deliberately front-loaded so it’s never a blocker later.
 - Terminal/command-line basics: navigating directories, creating/moving/deleting files, piping (`|`), redirects (`>`). If on Windows, install **WSL2** (Windows Subsystem for Linux) now — bioinformatics tooling in Phase 5 assumes a Linux-like shell, so this is worth doing early rather than retrofitting later.
 - **Git & GitHub:** install git, create a GitHub account, learn `init`, `add`, `commit`, `push`, `pull`, `clone`, and how to write a basic `.gitignore`. Do this via GitHub’s own “Hello World” guide plus just committing your CS50P exercises daily — real reps beat a tutorial here.
 - **Python environments:** install a Python distribution (Anaconda or Miniconda is the simplest for a beginner) and learn to create/activate a virtual environment (`conda create/venv`) per project. Also create a free **Google Colab** account now — you’ll need free cloud GPU access from Phase 4 onward, and it’s one less thing to figure out under time pressure later.
- **Math intuition (45 min/day, starting once the diagnostic/ramp above is done — around day 4–6):** 3Blue1Brown — “Essence of Linear Algebra” (YouTube, ~3 hrs total, free). Watch the whole series once, take rough notes, don’t force yourself to derive anything yet. Now that notation and basic algebra are fresh, this should land as genuinely intuitive rather than another wall of new symbols.

Weeks 3–4 (~2.75 hrs/day)

- **Python fundamentals, continued (1.25 hrs/day):** exceptions, file I/O, dictionaries/lists comprehensively, a little OOP. Supplement with *Automate the Boring Stuff with Python* (free online, automatetheboringstuff.com) for a second explanation in a more applied register wherever CS50P feels too abstract.
- **SQL (45 min/day): SQLBolt** — interactive, free, ~2–3 hrs total. Covers `SELECT`, `WHERE`, `JOIN`, `GROUP BY`, aggregate functions, subqueries at an intro level. Follow with the first half of the **Mode Analytics SQL Tutorial**.
- **Math intuition, continued (45 min/day):** 3Blue1Brown — “Essence of Calculus” (YouTube, ~5 hrs) — same treatment as linear

algebra. You're building visual intuition for vectors, matrices, derivatives, and gradients before you ever see them in an ML context.

Mini-project (last weekend of the month)

“Lab Notebook CLI” — a command-line Python program (no web/UI needed) that manages a small dataset via lists/dicts and reads/writes to a CSV file. - Spec: a program that lets a user (via terminal input) add, view, search, and delete entries in a simple dataset — frame it as a “sample tracker” (e.g., tracking lab samples: ID, date, sample type, result) since it’s familiar territory from microbiology. - Requirements: at least 4 functions, one dictionary-of-dictionaries or list-of-dictionaries data structure, CSV read/write persistence, basic error handling (e.g., invalid input doesn’t crash the program). - Stretch goal: add simple statistics (count entries by type, average of a numeric field) using only built-in Python (no pandas yet).

Gate checkpoint (must pass before Phase 1)

- ☐ Can write a function from scratch that takes a list of dicts and filters/aggregates it, without looking anything up.
- ☐ Can explain, in your own words, what a matrix multiplication represents and what an eigenvector is (no formulas needed — intuition is enough).
- ☐ Can write a SQL query combining SELECT, WHERE, JOIN, and GROUP BY from memory.
- ☐ Can commit and push a change to a GitHub repo from the terminal without looking up the commands.

Phase 1 — September 2026: Python Fluency + Data Tools + Stats Foundations

NYSC active · ~2.5–3 hrs/day · Weeks 5–8

Goal: Go from “can write basic scripts” to “comfortable manipulating real datasets.” This is where numpy/pandas/matplotlib enter, alongside real statistics.

Fixed gap: the original version of this phase stacked Python/data + SQL + math to 3–4.25 hrs/day against a 2.5–3 hr/day budget. Below, it’s split into two 2-week blocks so nothing is over-stacked on any given day.

Weeks 5–6 (~2.75 hrs/day): data stack + stats

- **Python → data stack (1.75 hrs/day):** finish any remaining CS50P material in the first couple of days if needed, then **Kaggle Learn** (all free, exercise-based, ~3–4 hrs each): **“Python”** (fast review/fill gaps)

→ “**Pandas**” → “**Data Visualization.**” Alongside, work through the official NumPy Quickstart and practice array indexing, broadcasting, and vectorized operations — this matters later for implementing ML from scratch.

- **Math (1 hr/day): Khan Academy — Statistics & Probability** track: descriptive stats, distributions, hypothesis testing basics, correlation vs. causation, conditional probability, Bayes’ theorem. This will bleed into Phase 2 — that’s expected.
- SQL is paused this block — you did enough in Phase 0 to not lose it in two weeks.

Weeks 7–8 (~2.75 hrs/day): SQL + cleaning + Project 0

- **SQL (45 min/day): SQLZoo** for more structured practice, then start **LeetCode’s “SQL 50”** study list, easy tier only — 2–3 problems/week. Set up SQLite locally (via Python’s built-in `sqlite3` or DB Browser for SQLite) and practice against a toy database you build yourself.
- **Math, continued (45 min/day):** keep working through Khan Academy stats/probability. Also add one topic Khan Academy doesn’t cover well but genomics leans on constantly: read a short applied explainer on **multiple hypothesis testing correction** (Bonferroni and false discovery rate/Benjamini-Hochberg) — a 15-minute blog-level read is enough at this stage; you’ll meet it again properly in Phase 5.
- **Data cleaning + Matplotlib/Seaborn (1.25 hrs/day):** Kaggle Learn’s “**Data Cleaning**” micro-course, plus enough Matplotlib/Seaborn to confidently make and label line, bar, scatter, histogram, and boxplots.

Mini-Project 1.1 — confidence-builder (1 evening, alongside the SQL block above)

“**Lab Sample Database**” - **Purpose:** turn the SQL you just learned into muscle memory against data you already know, before Project 0 asks for something bigger. - **Spec:** take the CSV your Phase 0 “Lab Notebook CLI” produces (or generate a synthetic version with ~200 rows if you didn’t keep the original), load it into a local SQLite database with 2 related tables (e.g., `samples` and `results`, linked by a sample ID) instead of one flat table. - **Deliverables:** write and save 8–10 queries covering: a multi-table JOIN, a `GROUP BY` with an aggregate, a subquery, and a window function (e.g., ranking samples by a result value within a group). Save them as a `.sql` file with a one-line comment above each explaining what it answers. - **Stretch goal:** wrap 2–3 of the queries in a tiny Python script using `sqlite3` that prints a formatted report — this previews the Python+SQL combination you’ll use constantly later.

Project 0 — foundational, low-stakes (weekends of weeks 7–8)

“**Exploratory Data Analysis Sprint**” - **Purpose:** first full contact with a real dataset, low pressure, purely about pandas/matplotlib fluency and commu-

nicating findings — not modeling. - **Dataset:** pick one small, clean, biology-adjacent dataset. Good options: WHO Life Expectancy dataset (Kaggle), a small clinical dataset like the Pima Indians Diabetes dataset (Kaggle/UCI), or a public COVID case-count dataset. - **Deliverables:** 1. Load data with pandas, inspect shape/dtypes/missing values/summary stats. 2. Clean it: handle missing values (document your reasoning for each choice), fix dtypes, remove/flag outliers. 3. Produce 4–6 visualizations that each answer a specific question (e.g., “does X correlate with Y?”, “how does Z vary across groups?”). 4. Write a short (300–500 word) markdown summary of 3 concrete insights, written for a non-technical reader. 5. Push to GitHub with a clean README (what the dataset is, what questions you asked, what you found, how to run the notebook). - **Stretch goal:** compute and interpret a correlation matrix + one basic hypothesis test (e.g., t-test between two groups) using `scipy.stats`, and explain the p-value correctly in the README.

Gate checkpoint (must pass before Phase 2)

- ☐ Can take a messy real CSV and produce a clean pandas DataFrame with correct dtypes and documented missing-value handling, without a tutorial open.
- ☐ Can explain mean/median/variance/standard deviation, and what a p-value does and does not mean.
- ☐ Can write a SQL query with a subquery or a multi-table JOIN unaided.

Phase 2 — Early/Mid October 2026: The Transition (Stats Deepening + ML Theory Entry)

NYSC winds down · Weeks 9–10 tight, then ramping to full-time

This is a short, deliberately lighter “bridge” phase — NYSC is ending, your schedule is in flux, and you’re about to step up to full-time study. Use it to consolidate stats and take your very first real steps into ML theory, rather than launching a new major topic.

Fixed gap: Andrew Ng’s Course 1 alone is realistically ~25–35 hours of video + exercises, which does not fit into a 2-week bridge phase on top of everything else. It’s fine — and expected — for Course 1 to spill 3–5 days into the start of Phase 3. Don’t rush it just to hit an arbitrary phase boundary; the gate checkpoint below is what actually matters, not the calendar.

Also: Coursera’s audit-mode policy has shifted over time (sometimes it’s full free video access, sometimes graded material gets paywalled). Check the current audit terms when you get here — if video-only access is no longer free, **Stanford’s original CS229 lecture notes**

and problem sets (publicly available, free) plus StatQuest cover the same ground and are a fine substitute.

Week 9 (~3 hrs/day, still NYSC-constrained)

- **Math (1 hr/day):** finish Khan Academy Probability & Statistics entirely.
- **ML theory entry (1.5 hrs/day):** start Andrew Ng’s Machine Learning Specialization, Course 1 (Supervised Machine Learning: Regression and Classification) — watch lectures, code along where possible. Pair dense topics with the matching **StatQuest** video first if a concept isn’t landing.
- **SQL (30 min/day):** window functions and **CASE WHEN** via Mode Analytics’ advanced tutorial section.

Week 10 onward (NYSC over, ramping to ~5–6 hrs/day)

- **ML theory (continue):** finish Course 1, including the exercises — don’t skip these, they’re where the actual learning happens.
- **Math:** Khan Academy **Multivariable Calculus**, but only the gradients/partial-derivatives sections — that’s the part ML actually leans on. Start keeping “**Mathematics for Machine Learning**” (free book, Deisenroth/Faisal/Ong, freely available PDF) on hand as a *reference*, not a cover-to-cover read.
- **SQL:** correlated subqueries, plus 10–15 medium-difficulty problems on **StrataScratch** or LeetCode SQL.

Mini-Project 2.1 — confidence-builder (~3–4 hrs, end of Week 10)

“**Linear Regression from Scratch**” - **Purpose:** the first project where you build the *algorithm*, not just the pipeline around it — this is what makes gradient descent click permanently instead of staying an abstract diagram. - **Spec:** numpy only, no scikit-learn. Generate or download a small single-feature or two-feature dataset (e.g., a simple synthetic linear relationship with noise, or a small real dataset like height-vs-weight). - **Deliverables:** implement the cost function (MSE), the gradient computation, and the update loop by hand; plot the cost decreasing over iterations to confirm convergence; plot your fitted line against the data; compare your learned coefficients against `np.linalg.lstsq` or scikit-learn’s **LinearRegression** on the same data and report how close they are. - **Stretch goal:** implement logistic regression from scratch the same way (sigmoid + binary cross-entropy loss instead of MSE) on a small 2-class synthetic dataset — this directly previews Project 1’s core model family.

Gate checkpoint (must pass before Phase 3)

- ☐ Can explain gradient descent (cost function, gradient, learning rate, convergence) without notes.

- ☐ Have a working from-scratch linear regression implementation in a script, not copied from a tutorial.
 - ☐ Comfortable with SQL window functions (e.g., can write a query ranking rows within groups).
 - ☐ Understand train/test split and *why* you don't evaluate on training data.
-

Phase 3 — November 2026: Classical ML, Full-Time + Project 1

Full-time · ~5–6 hrs/day

Goal: Real fluency with the classical ML toolkit — the algorithms, when to use them, how to evaluate them properly, and how to build a full pipeline end-to-end. This is the phase where you become “dangerous” with scikit-learn.

Core ML (3–3.5 hrs/day)

- Continue/finish Ng’s Specialization: Course 2 (Advanced Learning Algorithms — neural nets intro, decision trees) and Course 3 (Unsupervised Learning, Recommenders, Reinforcement Learning basics — skim this one, it’s not central to your near-term goals).
- Alongside, via StatQuest + hands-on practice, make sure you deeply understand: logistic regression, decision trees, random forests, gradient boosting (XGBoost/LightGBM conceptually), k-means clustering, PCA, k-NN, SVMs (conceptually — don’t over-invest here), regularization (L1/L2), and the bias-variance tradeoff.
- **Evaluation literacy** (this matters enormously for biomedical data, which is often imbalanced): precision, recall, F1, ROC-AUC, PR-AUC, confusion matrices, cross-validation (k-fold, stratified k-fold).
- **Fixed gap — class imbalance handling:** the original plan taught the right metrics for imbalanced data but never taught what to *do* about the imbalance itself. Add: class weighting (`class_weight='balanced'` in scikit-learn), oversampling/undersampling, and SMOTE (via the `imbalanced-learn` library). Biomedical datasets (disease/AMR prediction especially) are very often imbalanced — this is not optional knowledge for Project 1.

Practice (1.5–2 hrs/day)

- **Kaggle Learn:** “Intro to Machine Learning,” “Intermediate Machine Learning,” “Feature Engineering,” and “Intro to SQL” / “Advanced SQL” (formalizes what you already know).
- **Fixed gap — this block was overloaded:** do at most **one** small guided Kaggle “Getting Started” competition (Titanic or similar) as a

single warm-up rep, not 2–3. The real practice is Project 1 itself; extra competitions here just eat into the time it needs.

Mini-Project 3.1 — confidence-builder (~1 day)

“**Titanic Warm-Up**” - **Purpose:** one full rep of the classical ML pipeline shape (load → clean → engineer → train → evaluate → submit) on a forgiving, well-documented dataset, so Project 1 isn’t your first time touching every stage at once. - **Spec:** the Kaggle “Titanic — Machine Learning from Disaster” competition (or an equivalent small guided competition). Don’t chase leaderboard rank — the goal is completing the full loop once, cleanly. - **Deliverables:** a notebook covering EDA → missing-value handling → at least one engineered feature → two model comparisons (e.g., logistic regression vs. random forest) → a Kaggle submission. Keep it short; this is a rep, not a portfolio piece.

Project 1 — Classical ML, biology-flavored (last 2.5–3 weeks of the month, ~50–60 hrs)

“Antimicrobial Resistance / Disease Classification from Tabular Biomedical Data”

- **Goal:** a full, professional classical-ML pipeline on a real biomedical dataset, leaning into your microbiology background.
- **Dataset options (pick one, or another with similar structure):**
 - UCI “Breast Cancer Wisconsin (Diagnostic)” dataset — clean, good for a first full pipeline.
 - A public antimicrobial resistance (AMR) dataset — e.g., PATRIC/BV-BRC AMR datasets, or a curated AMR prediction dataset on Kaggle (search “antibiotic resistance prediction dataset”). This is the strongest choice for narrative fit with your background.
 - Kaggle “Heart Disease UCI” dataset as a fallback if AMR data proves hard to source cleanly.
- **Pipeline requirements (each stage should be its own clearly labeled notebook section or script):**
 1. **EDA:** class balance, feature distributions, missingness, correlation structure.
 2. **Cleaning & preprocessing:** justified handling of missing data, encoding categoricals, scaling where needed.
 3. **Feature engineering:** at least 2–3 engineered or selected features beyond the raw columns, with reasoning documented. 3b. **Imbalance handling (if applicable):** if your chosen dataset has meaningful class imbalance, explicitly address it — class weighting or SMOTE — and document before/after evaluation so the effect is visible, not just asserted.
 4. **Baseline model:** simple logistic regression or decision tree, honestly evaluated — this is your reference point.

5. **Model comparison:** train and compare at least 3 model families (e.g., logistic regression, random forest, XGBoost) using consistent, stratified cross-validation.
 6. **Hyperparameter tuning:** `GridSearchCV` or `RandomizedSearchCV` on your best-performing model family.
 7. **Evaluation:** report metrics appropriate to the class balance (don't just report accuracy on an imbalanced dataset — use precision/recall/ROC-AUC/PR-AUC as appropriate), with a confusion matrix and ROC curve plotted.
 8. **Interpretability:** feature importances (built-in or permutation-based) and, ideally, a SHAP summary plot explaining what's driving predictions — this is a strong differentiator on a biomedical project.
 9. **Write-up:** a clean GitHub README covering: problem statement, dataset description and source, methodology, results table, key findings in plain language, and limitations (be honest about what the model can't tell you — this reads as maturity to anyone evaluating it).
- **Stretch goals:** deploy the final model behind a minimal Streamlit or Gradio app where a user can input feature values and get a prediction + explanation; or write a short blog-style post walking through the biological reasoning behind your feature engineering choices.

Gate checkpoint (must pass before Phase 4)

- ☐ Can explain, for a given dataset, which evaluation metric matters and why — not just “I used accuracy.”
- ☐ Understand what overfitting looks like in a learning curve and what regularization/cross-validation does about it.
- ☐ Project 1 is complete, on GitHub, with a README a stranger could understand in 2 minutes.

Phase 4 — December 2026: Deep Learning Foundations + Project 2

Full-time · ~5–6 hrs/day

Goal: Understand neural networks from first principles (not just `.fit()`), get fluent in PyTorch, and build a real deep learning project — ideally image-based, since that maps naturally onto a microbiology background (microscopy, cell imaging).

Fixed gap: the original version of this phase asked for the *full* fast.ai course, the *full* Karpathy Zero to Hero series (micrograd through a GPT build), a PyTorch primer, and Project 2 — all inside one month. Realistically that's 2+ months of content. Below, fast.ai is

the primary spine, Karpathy is trimmed to a single targeted video, and raw PyTorch gets dedicated time because Project 2 needs it directly.

Weeks 1–2 (~3 hrs/day): primary spine + backprop intuition

- **fast.ai** — “**Practical Deep Learning for Coders**” **Part 1** (free), lessons 1–4: top-down approach, you’re training real models by lesson 1. This is your main spine for the month — follow it through image classification.
- **One targeted Karpathy video, not the full series:** “*The spelled-out intro to neural networks and backpropagation: building micrograd*” (~2.5 hrs, YouTube, free). This single video gives you the from-scratch backprop intuition that matters most; treat the rest of Zero to Hero (makemore, GPT) as optional future material once your projects are done, not a Month 4 requirement. Use an LLM as an active tutor here — ask it to explain any line you don’t follow, or quiz you on the material.
- **Compute note:** fast.ai’s own notebooks run on cloud GPUs by default, but for everything else this month — including Project 2 — use **Google Colab’s free GPU tier** (set up in Phase 0) or Kaggle Notebooks. Training CNNs on a CPU-only laptop is either painfully slow or infeasible for some of what’s below; don’t try to push through on CPU.

Weeks 3–4 (~3 hrs/day): raw PyTorch + Project 2

- **Fixed gap — raw PyTorch muscle memory:** fast.ai’s `fastai` library is high-level and abstracts away exactly the things Project 2 requires you to write by hand (a custom `Dataset/DataLoader`, a manual training loop). Before starting Project 2, do the official PyTorch **60 Minute Blitz**, then complete the mini-project below as a bridging exercise.
- **Math as-needed (0.5–1 hr/day, folded into this block):** chain rule and backprop math, softmax, cross-entropy loss — pull from 3Blue1Brown’s neural network series and the Mathematics for ML book as specific gaps come up.

Mini-Project 4.1 — confidence-builder (~1 day)

“**Raw PyTorch Bridge**” - **Purpose:** the gap between “fast.ai trained a model for me” and “I can write the training loop myself” is exactly where people quietly stall in Month 4 — this closes it deliberately before Project 2 needs it. - **Spec:** rebuild fast.ai’s lesson-1 image classifier (or an equivalently simple image classification task, e.g., MNIST or a small 2-class subset of any dataset) using **raw PyTorch only** — no `fastai` library. - **Deliverables:** a custom `Dataset/DataLoader`, a model defined via `nn.Module`, a manual training loop with the optimizer step written out explicitly, and a training/validation accuracy plot over epochs.

Project 2 — Deep Learning, image-based, biology-flavored (weeks 3–4, running alongside the above, ~50–60 hrs)

“Microscopy Image Classification with Transfer Learning”

- **Goal:** a real CNN-based deep learning project on biological image data, with proper training practice and model interpretability — a strong, distinctive portfolio piece given your background.
- **Dataset options:**
 - NIH/Kaggle “**Malaria Cell Images Dataset**” (parasitized vs. uninfected blood smear cells) — excellent fit, well-documented, manageable size.
 - A bacterial colony image classification dataset (search Kaggle for “bacteria colony classification” or “DIBaS dataset” — Digital Image of Bacterial Species).
 - Chest X-ray pneumonia classification dataset as a fallback if a microbiology-specific dataset proves hard to source cleanly.
- **Pipeline requirements:**
 1. **Data pipeline:** custom `Dataset/DataLoader` in PyTorch, with appropriate train/val/test splits and data augmentation (flips, rotations, color jitter as appropriate — justify choices given the imaging modality).
 2. **Baseline:** a small CNN trained from scratch, honestly evaluated, to establish a reference point.
 3. **Transfer learning:** fine-tune a pretrained model (ResNet18/34 is a good size) on the dataset — compare frozen-backbone vs. full fine-tuning.
 4. **Training discipline:** proper training loop with train/val loss and accuracy tracked and plotted per epoch, early stopping or checkpointing on best val performance, learning rate scheduling if time allows.
 5. **Evaluation:** confusion matrix, precision/recall/F1 (per-class if multi-class), and a handful of example predictions (correct and incorrect) visualized.
 6. **Interpretability:** Grad-CAM visualizations showing what regions of the image the model is attending to — this is both scientifically interesting and a strong signal of rigor to anyone reviewing the project.
 7. **Write-up:** GitHub README with problem statement, dataset/source, architecture and training details, results, Grad-CAM examples with brief interpretation, and limitations.
- **Stretch goals:** wrap the trained model in a minimal Gradio/Streamlit demo where a user uploads an image and gets a prediction + Grad-CAM overlay; or benchmark 2 different pretrained architectures (e.g., ResNet vs. EfficientNet) and discuss the tradeoffs.

Gate checkpoint (must pass before Phase 5)

- ☐ Can explain backpropagation and gradient descent for a neural net without hand-waving (you should be able to sketch it on paper).
 - ☐ Comfortable writing a PyTorch training loop from a blank file, not copy-pasted.
 - ☐ Project 2 complete, on GitHub, with a proper README and at least one interpretability visualization.
-

Phase 5 — January 2027: The Bioinformatics Bridge + Project 3 + Portfolio

Full-time · ~5–6 hrs/day

Goal: This is the phase that actually makes you *bioinformatics*-ready, not just ML-generalist-ready. You'll pick up sequence data handling, the tools your BSc Hons program will expect (R/Bioconductor at a working level), and build the capstone project that ties biology and ML together explicitly.

Fixed gap — this is the most important fix in the whole plan. The original version of this phase taught sequence *file formats* and *ML on sequences*, but never taught the actual command-line bioinformatics ecosystem — the tools a real lab runs on daily (alignment, variant calling, database querying, pipeline tools). Without this, “lab-ready” wasn’t really true. It’s added below as its own block.

Core bioinformatics command-line toolkit (1–1.5 hrs/day, weeks 1–2 of the month)

- **Unix text-processing on biological files:** practice `grep`, `awk`, `sed`, and pipes directly on FASTA/FASTQ files — e.g., count reads, extract sequences matching a pattern, compute basic length stats from the command line before ever opening Python. This is a real, constantly-used skill in bioinformatics work.
- **samtools:** install it, work through basic operations on a small BAM/SAM file (view, sort, index, basic stats) using a small public example file.
- **BLAST:** run both the NCBI web BLAST interface and the command-line `blastn`/`blastp` tools against a small local database, to understand what sequence similarity search actually does and why it’s foundational to so much of the field.
- **Querying public databases programmatically:** Biopython’s `Bio.Entrez` module to query NCBI directly from Python (fetch a sequence or record by accession number) — this connects your Python skills directly to real bioinformatics data access.

- **Pipeline tools, conceptually:** you don’t need to build a production pipeline in one week, but read a short introduction to **Snakemake** (or Nextflow) and understand *why* workflow managers exist in bioinformatics (reproducibility, dependency management across multi-step analyses) — you’ll very likely meet one of these in your program.

Mini-Project 5.1 — confidence-builder (~1–2 days)

“**Command-Line Sequence Toolkit**” - **Purpose:** consolidate the command-line block above into one small, real artifact instead of letting it stay a set of disconnected exercises. - **Spec:** given a small public FASTQ/FASTA file and a small BAM file (any suitable public example, e.g., from a tutorial dataset on the samtools or Biopython documentation), write a short bash script (or Python script shelling out to the tools) that: counts total reads, extracts sequences matching a simple pattern via **grep/awk**, runs a basic samtools stats command on the BAM file, and runs one command-line BLAST search against a small local database. - **Deliverables:** the script itself plus a short **README.md** explaining what each step does and why — written as if handing it to a labmate who’s never used the command line before. This doubles as documentation practice.

Bioinformatics fundamentals (1–1.5 hrs/day, weeks 2–3)

- File formats and core concepts: FASTA, FASTQ, BAM/SAM, VCF — what they are, why they exist, how to inspect them. A short applied primer like Rosalind.info’s “Bioinformatics Stronghold” problem set (free, gamified — genuinely good for this) is a strong way to learn this by doing rather than reading.
- **Biopython:** work through the official Biopython tutorial’s core chapters (sequence objects, parsing FASTA/FASTQ, simple sequence manipulation).

R primer (30–45 min/day, lower priority — cut first if behind schedule)

- **Honest framing:** a few hours of R now will not make you fluent in the Bioconductor ecosystem (DESeq2, limma, and friends) that your program will actually use for things like RNA-seq differential expression — that’s realistically learned properly *during* the program, with an instructor and real coursework. What’s worth doing now is just enough that R syntax and the RStudio environment aren’t a first-day shock.
- Install R + RStudio. Work through the free online book “*R for Data Science*” (Hadley Wickham, r4ds.hadley.nz) selectively — data import, dplyr, ggplot2.
- If your program’s syllabus is available and lists specific Bioconductor packages, install them and skim one vignette (DESeq2’s is a good general example) just to see the shape of a typical Bioconductor workflow — including

where multiple-testing correction (Phase 1's FDR/Bonferroni note) shows up in practice, since it's central to differential expression analysis.

ML on sequences (2–2.5 hrs/day)

- Learn the standard approaches to representing biological sequences for ML: k-mer frequency encoding for classical ML, and one-hot/embedding encoding for deep learning approaches.
- Build up to sequence classification: start with k-mer + classical ML (fast, interpretable, good baseline), then optionally a simple 1D-CNN or small RNN/LSTM on one-hot-encoded sequences for comparison.

Project 3 — Bioinformatics capstone (bulk of the month, ~70–90 hrs)

“Sequence-Based Prediction: choose one lane and go deep”

Pick **one** of the following (all specified in enough detail to hand to another AI or execute solo):

Option A — Antimicrobial Resistance Gene Prediction from Sequence Data - Task: given bacterial genomic sequences (or gene sequences), predict presence/class of resistance genes. - Data: PATRIC/BV-BRC, NCBI's AMR gene database, or curated Kaggle AMR sequence datasets. - Pipeline: sequence QC → k-mer feature extraction (e.g., k=4 to 6) → classical ML baseline (random forest/XGBoost) → compare against a simple CNN on one-hot encoded sequences → evaluate with precision/recall/AUC given likely class imbalance → interpret which k-mers/motifs matter most (feature importance on the classical model).

Option B — Protein Family / Function Classification - Task: classify protein sequences into functional families. - Data: a subset of UniProt or Pfam-seed sequences (curate a manageable subset — e.g., 5–10 well-separated families, a few thousand sequences total, to keep training tractable). - Pipeline: same structure as Option A, but on protein sequences (20-letter alphabet instead of 4-letter DNA alphabet). Stretch goal: try a pretrained protein language model embedding (e.g., ESM-2, small variant) as input features to a simple classifier, and compare against your from-scratch k-mer baseline — this is an ambitious but genuinely impressive stretch goal.

Option C — Drug Response Prediction - Task: predict cell-line sensitivity/resistance to a drug from gene expression or genomic features. - Data: GDSC (Genomics of Drug Sensitivity in Cancer) public dataset. - Pipeline: feature selection/dimensionality reduction (this dataset is high-dimensional — PCA or feature selection is essential and a good demonstration of judgment) → regression or classification model → evaluate with appropriate regression metrics (RMSE, R^2) or classification metrics → interpret which genes/features drive predicted response.

Common requirements across all options: 1. Proper train/val/test methodology with no data leakage (a common and serious mistake in genomics ML — document explicitly how you avoided it, e.g., splitting by sequence similarity/cluster rather than randomly if relevant). 2. At least one classical ML model and one deep learning approach, compared honestly. 3. Biologically-informed interpretation of results, not just metrics — what do the important features/regions/motifs suggest, in plain language? 4. A GitHub README written as if for a lab rotation or thesis committee: problem, biological motivation, data, methods, results, limitations, and a clearly stated “what I’d try next with more time/data” section (this signals research maturity, which matters a lot for what you’re heading into).

Portfolio & professional polish (parallel, 1 hr/day throughout the month)

- Clean up all three project READMEs to a consistent, professional standard.
- Build a simple GitHub Pages or Markdown-based portfolio site linking all three projects with 2–3 sentence summaries each.
- Update/create a resume/CV framed around: Python, SQL, classical ML, deep learning (PyTorch), bioinformatics tooling (Biopython, sequence data, R basics), and the three projects as evidence.
- Git/GitHub hygiene: consistent commit history (not one giant commit), `.gitignore`, `requirements.txt` or `environment.yml` for every project so anyone can reproduce your environment.

Gate checkpoint (must pass before Phase 6)

- ☐ Comfortable explaining k-mer encoding and why sequence data needs different handling than tabular data.
- ☐ Can read a FASTA/FASTQ file and explain what’s in it without looking it up.
- ☐ Can run a basic samtools command and a command-line BLAST search without hand-holding.
- ☐ Project 3 complete, README written to “lab-ready” standard.
- ☐ All three projects live on GitHub with a portfolio page linking them.

Note: this is intentionally the heaviest, most content-dense phase in the plan, because it’s the one that closes the biggest original gap. If it’s genuinely tight, the R primer is the piece to compress or defer — the command-line toolkit and Project 3 are the load-bearing parts.

Phase 6 — Early February 2027: Consolidation & Departure Prep

Full-time, ~2–3 weeks · Lighter, deliberately

Goal: No new major material. This phase is about tightening what you have, being able to talk about it fluently, and arriving in South Africa ready rather than exhausted.

- **Interview-style practice:** 30–45 min/day of LeetCode (Python, easy/medium) and SQL practice, purely to stay sharp — not to cram new topics.
- **Project storytelling:** for each of the 3 projects, prepare a 2-minute verbal walkthrough (what, why, how, what you’d do differently) — practice saying it out loud, not just having it written down. This is what actually gets used in interviews and supervisor conversations.
- **Syllabus reconnaissance:** if your BSc Hons program’s syllabus or reading list is available, skim it now and patch any obvious gaps (a specific R package, a specific statistical test, a specific tool like a genome browser) while you still have full-time hours to do it.
- **Rest:** deliberately taper hours in the final week before travel. Arriving well-rested and confident is worth more than one more course squeezed in.

Ongoing / Parallel Habits (all 7 months, every phase)

- **Coding fluency:** 1 LeetCode-style problem, 3x/week (Python, easy → medium as you progress). NeetCode.io has free, well-structured lists. This isn’t about becoming a software engineer — it’s that coding fluency compounds and quietly makes every project faster.
- **Field literacy:** read 1 ML or bio-ML blog post or paper abstract per week, even before you fully understand it. This builds vocabulary and pattern recognition over time — by Phase 3–4 you should notice papers getting easier to parse.
- **Weekly log:** 5 bullet points, every week, no exceptions — what you learned, what you built, what confused you. This is your progress record and future CV material.
- **Community touchpoints:** an active Kaggle profile (even just notebooks and small competition entries), and following a handful of bio-ML people/labs online — this builds context for where the field is heading and surfaces vocabulary/tools before you hit them formally.
- **Fixed gap — don’t self-teach in total isolation:** join at least one active community where you can ask questions when genuinely stuck (Bioinformatics Stack Exchange, r/bioinformatics, or a Kaggle discussion forum are all free and low-friction). Self-teaching solo for 7 months is a real burnout and stuck-point risk that a working roadmap should account for,

not just assume away.

Resource Library (everything in one place, by category)

Everything referenced above, consolidated for quick reference. All free unless marked otherwise.

Programming & tooling - CS50P — Python fundamentals, autograded, Phase 0. - *Automate the Boring Stuff with Python* (automatetheboringstuff.com) — applied Python reference, Phase 0. - GitHub “Hello World” guide — git/GitHub basics, Phase 0. - Anaconda/Miniconda — Python distribution + environment management, Phase 0. - Google Colab — free cloud GPU notebooks, set up Phase 0, used heavily from Phase 4. - WSL2 (Windows only) — Linux shell on Windows, Phase 0.

SQL - SQLBolt — interactive intro, Phase 0. - Mode Analytics SQL Tutorial — intro + advanced (window functions, subqueries), Phases 0 & 2. - SQLZoo — structured practice, Phase 1. - LeetCode “SQL 50” study list — interview-style practice, Phases 1 & 6. - StrataScratch — medium-difficulty SQL problems, Phase 2. - DB Browser for SQLite — GUI for local practice databases, Phase 1.

Math - Khan Academy — Algebra 1/2 diagnostic + targeted units, Phase 0 ramp. - Khan Academy — Precalculus (“Functions” unit), Phase 0 ramp. - 3Blue1Brown — “Essence of Linear Algebra” (YouTube), Phase 0. - 3Blue1Brown — “Essence of Calculus” (YouTube), Phase 0. - Khan Academy — Statistics & Probability, Phases 1–2. - Khan Academy — Multivariable Calculus (gradients/partial derivatives sections only), Phase 2. - *Mathematics for Machine Learning* (free PDF, Deisenroth/Faisal/Ong) — reference throughout, not cover-to-cover. - 3Blue1Brown — neural network / backprop series, Phase 4.

Data manipulation & classical ML - Kaggle Learn micro-courses: Python, Pandas, Data Visualization, Data Cleaning, Intro to SQL, Advanced SQL, Intro to Machine Learning, Intermediate Machine Learning, Feature Engineering — Phases 1 & 3. - NumPy Quickstart — Phase 1. - Matplotlib / Seaborn documentation — Phase 1. - Andrew Ng’s Machine Learning Specialization (Coursera, audit) — Phases 2–3. Fallback: Stanford CS229 lecture notes (free) if audit terms change. - StatQuest with Josh Starmer (YouTube) — plain-English companion throughout Phases 2–3. - scikit-learn documentation — Phase 3. - **imbalanced-learn** library docs (SMOTE, class weighting) — Phase 3. - SHAP documentation — interpretability, Phase 3.

Deep learning - fast.ai — Practical Deep Learning for Coders Part 1 — primary DL spine, Phase 4. - Andrej Karpathy — “building micrograd” (YouTube) — targeted backprop intuition, Phase 4. - PyTorch 60 Minute Blitz — Phase 4. -

PyTorch official documentation — Phase 4. - **pytorch-grad-cam** (or equivalent) — interpretability for CNNs, Phase 4.

Bioinformatics - Rosalind.info — “Bioinformatics Stronghold” problem set, Phase 5. - Biopython tutorial & documentation (incl. **Bio.SeqIO**, **Bio.Entrez**) — Phase 5. - samtools documentation — Phase 5. - NCBI BLAST (web + command-line docs) — Phase 5. - Snakemake documentation/tutorial — Phase 5. - *R for Data Science* (free, r4ds.hadley.nz) — light R primer, Phase 5. - DESeq2 vignette (Bioconductor) — example of a real Bioconductor workflow, Phase 5.

Datasets - UCI Machine Learning Repository — general + Breast Cancer Wisconsin, Phase 3. - Kaggle: Malaria Cell Images, Heart Disease UCI, DIBaS (bacterial colonies), Titanic — Phases 3–5. - PATRIC/BV-BRC, NCBI AMR gene database — Phase 5 (AMR option). - UniProt / Pfam — Phase 5 (protein classification option). - GDSC (Genomics of Drug Sensitivity in Cancer) — Phase 5 (drug response option).

Career & portfolio - GitHub Pages — portfolio hosting, Phase 5. - NeetCode.io — structured LeetCode practice, ongoing. - Bioinformatics Stack Exchange, r/bioinformatics — community support, ongoing.

Project Library (every project, mini and main, in one place)

Full specs live in the phase sections above; this is the navigational index — what exists, where, and why. Difficulty increases top to bottom.

#	Project	Phase	Tier	Purpose
0.1	Lab Notebook CLI	Phase 0	1 — first rep	First real Python program: functions, data structures, CSV persistence, error handling.
1.1	Lab Sample Database	Phase 1	1 — first rep	Turns fresh SQL knowledge into muscle memory: JOINS, aggregates, subqueries, window functions.

#	Project	Phase	Tier	Purpose
0	Exploratory Data Analysis Sprint	Phase 1	2 — applied	First full contact with a real dataset: pandas, cleaning, visualization, written insight — no modeling yet.
2.1	Linear Regression from Scratch	Phase 2	2 — applied	Build the algorithm itself (cost function, gradient descent) in numpy — makes the theory permanent.
3.1	Titanic Warm-Up	Phase 3	2 — applied	One full rep of the classical ML pipeline shape on a forgiving dataset before Project 1 raises the stakes.
1	AMR / Disease Classification	Phase 3	3 — capstone	Full classical ML pipeline on real biomedical tabular data: EDA → modeling → tuning → interpretability.
4.1	Raw PyTorch Bridge	Phase 4	2 — applied	Closes the gap between “fast.ai trained it for me” and “I can write the training loop myself.”
2	Microscopy Image Classification	Phase 4	3 — capstone	CNN + transfer learning on real biological image data, with Grad-CAM interpretability.

#	Project	Phase	Tier	Purpose
5.1	Command-Line Sequence Toolkit	Phase 5	2 — applied	Consolidates core bioinformatics command-line tools (grep/awk, samtools, BLAST) into one working artifact.
3	Sequence-Based Prediction (AMR genes / protein family / drug response — pick one)	Phase 5	3 — capstone	The bioinformatics-ML bridge project: sequence representation, classical + deep approaches, biological interpretation.

Reading this table: Tier 1 projects exist purely to build confidence with a brand-new skill in isolation — they should feel almost easy. Tier 2 projects are where you start combining skills under light real-world constraints. Tier 3 projects are the ones that go on your CV and get discussed in interviews or with a prospective supervisor — each should be defensible in detail, not just demoable.

Dos and Don'ts

Do: - Timebox rabbit holes. Give a confusing tangent 20–30 minutes, then move on and come back to it later with fresh eyes if it's still unresolved. - Use an LLM as an active tutor — ask “why,” ask for a counter-example, ask it to quiz you — rather than just asking it for the answer and moving on. - Finish before you polish. Get every project “done and a bit ugly” before you spend time making it “done and pretty.” A finished small project beats an abandoned ambitious one, always. - Take the gate checkpoints seriously and honestly. If you're faking a pass, the next phase will make that expensive, not free. - Keep the weekly log even — especially — on the weeks that felt bad. Those are the ones you'll want the record of later. - Treat stretch goals as a menu, not a checklist. They exist so there's always a next step for a good week, not so every week needs one.

Don't: - Don't compare Month 1 you to the polished portfolios and highlight-reel progress you'll see online. Public ML content is heavily survivorship-biased; you're seeing years of someone's work compressed into a highlight reel. - Don't

chase every new tool or paper that trends while you're mid-plan. The field moves fast, but fundamentals age far better than whatever's hyped this month — stay the course. - Don't skip the "boring" parts of a project — evaluation, interpretability, the honest limitations section. They're what separates a junior engineer from someone who can only call `.fit()`. - Don't let perfectionism block shipping. A README that honestly states a project's limitations reads as more credible than a project that never gets pushed because it isn't perfect yet. - Don't try to complete every stretch goal in every project spec — that's scope creep dressed up as ambition, and it's the fastest way to blow a month's timeline. - Don't self-teach in total isolation for 7 straight months — use the community touchpoints. Getting unstuck in an hour beats getting stuck for three days out of pride. - Don't skip rest days to "catch up." A burnout in Month 4 costs far more time than any single rest day ever saves.

A Word of Encouragement

Worth saying plainly: what you're doing is genuinely hard, and the difficulty isn't a sign you're on the wrong path. You're changing fields, largely self-teaching, on a real deadline, often alongside NYSC obligations — that's a lot to be carrying at once, and it's normal for it to feel heavy in places, especially in Phase 0–1 before you have any real momentum yet.

The "am I actually behind" feeling is close to universal in self-taught technical paths — it shows up whether someone is on week 2 or year 2, and it is not a reliable signal that something is wrong. The gate checkpoints in this plan exist so you have something more honest than that feeling to check yourself against. Trust them over the anxiety.

Progress here compounds in a way that's hard to see week to week but very visible looking back a month at a time — re-reading your own Week 1 log in Month 5 is usually the moment this becomes obvious. You have a real scientific background, a real deadline forcing focus, and a plan built specifically for where you're starting from, not a generic one. That's a genuinely strong position to be building from.

Seven months of genuinely full-effort, well-sequenced study — most of it full-time — gets you to a real, defensible outcome: **someone who can competently join a bioinformatics lab or work as a junior ML engineer, with three real projects that demonstrate both technical range and biological judgment.** That is a strong, legitimate outcome, not a stretch fantasy — but it's also not "expert" or "senior," and it shouldn't be measured against that bar. Measure yourself month to month against the gate checkpoints: are you passing them without hand-waving, are your projects genuinely improving in rigor from Project 1 to Project 3, and could you defend each one in front of a

skeptical supervisor. That's the right yardstick, and by that yardstick this plan, executed honestly, gets you there.